

Trustworthy process monitor

Objective

Build an autonomous agent that automates the full data-reliability pipeline: it ingests a real, undocumented industrial sensor dataset, infers what the data represents, detects data quality issues and early signs of process drift, and diagnoses the root cause of any fault while keeping a human operator able to review, question, and override every conclusion, and without the raw data ever leaving the operator's environment. The same underlying approach should generalize beyond sensor data to other messy, high-volume data domains, such as manually entered business records.

Challenge Overview

Data breaks silently. A manufacturing sensor drifts out of calibration over weeks. A business system quietly accumulates thousands of small manual-entry errors. In both cases, by the time something visibly goes wrong, finding the root cause usually means hours of manual digging through dashboards and logs.

Participants will work with industrial sensor dataset as the primary testbed: a large volume of sensor readings across dozens of unlabeled columns and fault types, with no accompanying documentation. The system must independently make sense of this data, monitor it for signs of degradation, and produce a diagnosis an operator can trust and act on built in a way general enough to extend to other messy, high-volume data sources beyond sensors.

The system should:

- infer the likely meaning and role of each unlabeled sensor, grounded in evidence from the data itself
- check whether the incoming data itself can be trusted complete, in range, consistent before reasoning about the process
- turn plain-language operating rules into automated data quality checks
- detect drift from normal operating conditions toward a fault state
- identify the most likely fault type and rank the sensors responsible
- present a clear, step-by-step diagnosis for human review

Scope

The core analysis should cover:

- statistical profiling of each sensor (distribution, noise level, dynamic behavior) as the evidence base for any inference

- correlation and lagged cross-correlation analysis across sensors, since physically connected variables typically move together, often with a time delay
- a structural first pass distinguishing measured/output variables from manipulated/actuator variables, where such a distinction exists in the data
- use of general domain knowledge (common instrument types, typical unit operations such as reactors, separators, and strippers) to propose functional roles for clusters of related sensors as a hypothesis-generation aid, not a source of pre-determined answers
- data quality checks on the raw stream itself — missing values, frozen or stuck sensors, out-of-range readings, duplicate or gapped timestamps, inconsistent units flagged separately from process faults, since a broken sensor and a broken process look different and call for different actions
- translation of natural-language operating rules or thresholds into executable data quality checks
- detection of both abrupt anomalies and gradual drift toward one of the fault types
- attribution of any flagged event to specific signals, not an unexplained aggregate score
- ranking of sensors by contribution to a given fault, with a plain-language explanation of why each was implicated
- clear separation between what the system infers, what it assumes, and what remains uncertain
- data minimization by design: the LLM reasons over derived artifacts, statistical fingerprints, correlation summaries, rule text never raw sensor rows

Because the same underlying problem shows up across domains - a drifting sensor and a corrupted business record both look like an unexplained deviation from expected behavior — the rule-extraction, drift-detection, and root-cause logic should be designed for portability from the start, not added afterward. Participants should demonstrate this generalization, either through a working pass over a second, non-sensor dataset (for example a messy business or transactional dataset) or through a clearly reasoned architectural walkthrough of why the same logic would transfer.

Core Requirements

The system should:

- process the full sensor dataset without requiring the user to manually label columns
- ground every sensor inference in shown statistical or relational evidence (a specific correlation, a response lag, a distribution shape) rather than an asserted label — a

lower-confidence inference with visible reasoning should count as stronger than a confident-sounding label with none

- generate an initial data understanding report without requiring the user to write their own analysis prompt
- support configurable, human-authored rules that are converted into automated checks
- run baseline data quality checks (completeness, validity, consistency, timeliness) on every incoming batch before any drift or fault reasoning, and state explicitly when the data itself can no longer be trusted
- monitor incoming data for drift toward known fault types using sensor behavior only — fault labels and simulation metadata should inform evaluation, not the live detection pipeline
- trace every flag or alert back to the specific signal(s) responsible
- produce a root-cause diagnosis that includes fault type, contributing sensors, and confidence level
- allow a human operator to accept, question, or override any inference, flag, or diagnosis
- log all system decisions, the evidence behind them, and any human overrides for later review
- keep raw data inside the operator's environment; any call to an external model carries only derived summaries (statistical profiles, correlation matrices, plain-language rules), never raw records
- make the LLM layer swappable: the pipeline must run against a locally hosted or EU-hosted model through configuration, not a rewrite
- record every external model call: what was sent, to which model, and why
- demonstrate that the rule-extraction, drift-detection, and root-cause logic is not hard-coded to this dataset — through a second-domain pass, a reasoned architectural explanation, or both

Expected Output

1. Sensor understanding report: inferred identity/role for each unlabeled sensor, with the statistical or relational evidence behind each inference and a stated confidence level.
2. Automated data quality checks: baseline checks on completeness, validity, consistency, and timeliness, plus executable checks derived from plain-language rules, each with pass/fail status and traceability to the originating rule or check.

3. Drift and anomaly detection: continuous monitoring output showing deviation from normal operation, flagged where drift becomes significant, referencing the specific signal(s) responsible.
4. Root-cause diagnosis: fault type, ranked contributing sensors, and a step-by-step explanation suitable for an operator with no data science background.
5. Human-in-the-loop controls: a clear path for a human to review, accept, question, or override any system output.
6. Decision log: every inference, flag, and diagnosis, with supporting evidence, plus any human review or override.
7. Adaptability demonstration: evidence that the same pipeline logic applies beyond the sensor dataset, whether as a second-domain run (e.g. on messy business or transactional data) or a reasoned walkthrough of the underlying architecture.
8. Data-flow record: a short statement or diagram of what leaves the operator's environment, to which model, and why, plus evidence, the model layer can be swapped (a configuration toggle or a run against a local model).

Bonus Capabilities

- extending the adaptability demonstration to a third data domain even further from sensors, such as free-text or log-based records
- a "no-egress mode": the full pipeline running end-to-end on a locally hosted open-weight model
- confidence or uncertainty scores throughout, not just at the final diagnosis
- a natural-language interface for operators to ask "why" about any flag
- visual dashboards showing drift trends over time
- a critique/review step that challenges the system's own diagnosis before presenting it
- exporting the diagnosis and decision log as a shareable report

Evaluation

- End-to-end autonomy: the system independently understands the data, generates checks, and detects and diagnoses problems without manual labeling or hand-tuned prompts
- Data quality: the system detects when the data itself is broken (missing, stuck, out of range, inconsistent) and separates that from process faults, rather than diagnosing a "fault" that is really a dead sensor

- Human-in-the-loop design: every inference, flag, and diagnosis traces to specific evidence and is logged; a human operator can review, question, or override any output; a correct-sounding conclusion with no shown derivation should score lower than a well-reasoned, appropriately uncertain one
- Adaptability (sensors → business data) : the core rule-extraction, drift-detection, and root-cause logic generalizes beyond this dataset to other messy, high-volume data domains, including non-sensor business data
- Data control: raw data never leaves the operator's environment;

Note to Participants

This dataset has no ground-truth documentation. Some participants may recognize it or find related industrial-process material that knowledge may inform a hypothesis, but it doesn't substitute for evidence. Your system should show, from the data itself, why each inference was made. The goal is a decision-support tool that earns an operator's trust, not one that asserts conclusions it can't show its work for, and one built well enough that the same trust would carry over to a different kind of messy data entirely.

Datasets and Sources:

Tennessee Eastman Process Dataset (Harvard Dataverse): [Additional Tennessee Eastman Process Simulation Data for Anomaly Detection Evaluation - Harvard Dataverse](#)

Original Tennessee Eastman Process model by Downs & Vogel (1993)